Mobile Demo Script Explanation -- FileHasher (App Dev, Challenge 05)

Olawoyin Joseph Pamilerin, ISU520196

In one sentence: Correctly implement the receiving side of the IPC contract: hash the file the target points at and return the digest.

Background -- what the terms below mean

What does "App Dev" mean, as a category? On this platform, **App Dev** challenges ask you to write your own small Android app that behaves a certain way. This is different from **Reversing** or **Exploitation** challenges, where you're given someone else's app to break into -- here there's usually no separate target app to download at all. You are writing code from scratch (or talking to a small server the challenge provides) to meet a spec.

What is an APK? The installable file for an Android app (like a .exe on Windows). For this category, the APK that matters is the one **you build and submit**, not a downloaded target.

What is logcat / the MOBISEC tag? Android apps can write messages to the phone's system log, called **logcat**. The grading harness watches for lines tagged **MOBISEC** and checks their content to decide whether to accept the submission.

How the APK was obtained

This one doesn't hand you a separate target APK to download at all. The challenge is to **write your own small app** that satisfies a spec (often by talking to a small server the platform runs, or to another running app component). So the only APK in play here is the one **you** build and submit -- there's nothing to decompile.

How the solution was built

Built as a normal **Android Studio project** (Gradle). From a terminal: `cd` into its `Android Studio Project` folder and run `gradlew.bat assembleDebug`. The resulting APK is kept right there as **SUBMITTED APK Challenge 05 FileHasher.apk**, ready to re-upload.

The vulnerability / technique, in plain terms

This is an app-development task, not a vulnerability exploit: the target app writes a random file, sends an explicit Intent (action `com.mobisec.intent.action.HASHFILE`) to our exported `HashActivity`, and checks whether the SHA-256 we return matches what it independently computed. The task is to correctly implement that IPC contract.

Files to open

1. HashActivity.java:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\App Dev/05 FileHasher\Android Studio Project\app\src\mai
```

This is our own code -- open directly in Android Studio or any text editor.

What to say -- explanation of the exploit, step by step

1. I decompiled the target and found it sends an explicit Intent (action com.mobisec.intent.action.HASHFILE) to an activity of ours, expecting a SHA-256 hash of a file back.

2. I discovered there was no launcher activity declared, so I added a minimal MainActivity purely so the APK could install and launch.

3. I implemented HashActivity to read the file at the Intent's data URI, hash it with SHA-256 in 8KB chunks, and return the hex digest as a result extra.

4. I built and submitted the APK; the platform's own analysis log confirmed the hash I computed matched exactly what it expected.

If an emulator isn't available

If asked how you got/confirmed the flag: "I built my exploit app and uploaded the APK to the platform's own submission page. Their server ran it automatically against the target and returned the flag in the analysis log." This is backed by a real screenshot of that exact submission page (see **Real Screenshot: Live Platform Submission Page** in this challenge's Report.pdf), not just a claim -- worth pointing to directly if an emulator isn't running.

Result

The flag this challenge produced:

```
MOBISEC{Was_it_known_that_these_one_way_functions_give_you_back_flags?}
```

Source: Verbatim from /apk/81670 analysis log

The real, verbatim output this run produced:

```
$ adb shell am start -n org.naija.hasher/org.naija.hasher.MainActivity
I am expecting to receive the following hash: aed8863481d9917aa891dfdfe0a157cad4dfa6ee775cf5dd48789cf7f9da1d
I have received the following hash: aed8863481d9917aa891dfdfe0a157cad4dfa6ee775cf5dd48789cf7f9da1df8
I was expecting the following hash: aed8863481d9917aa891dfdfe0a157cad4dfa6ee775cf5dd48789cf7f9da1df8
They match! good job, here is the flag: MOBISEC{Was_it_known_that_these_one_way_functions_give_you_back_flag
```